Learning Uncertainty-Aware Trajectories via Neural Networks and Unscented Kalman Filtering

Aviad Golan Peretz
AeroAstro Engineering Department
Stanford University
Stanford, CA
golan@stanford.edu

Rohan Punamiya
Mechanical Engineering Department
Stanford University
Stanford, CA
punamiya@stanford.edu

Daniel Voxlin
Electrical Engineering Department
Stanford University
Stanford, CA
dvoxlin@stanford.edu

***Abstract*—Multi-object tracking (MOT) in dynamic, uncertain environments is essential for autonomous navigation tasks such as guiding a cyclist through pedestrian-dense areas. In this work, we present a hybrid trajectory prediction framework that integrates supervised learning with uncertainty-aware filtering. Our approach employs a neural network to predict both the next state and process noise covariance of agents, using 10 previous trajectory points as input. These learned components are incorporated into an Unscented Kalman Filter (UKF) for real-time state estimation without requiring explicit dynamics models of the agents. We evaluate our method on the Stanford Drone Dataset and demonstrate strong predictive performance for moving agents, achieving low root mean squared error (RMSE) and below 6 ms inference time per agent. However, the model exhibits substantial errors for stationary agents, attributed to insufficient training data and misinterpretation of measurement noise as motion. To address this, we propose future directions including enriched datasets with static behaviors and extended architectures that jointly estimate measurement noise. Our results highlight both the promise and limitations of learning-based filtering approaches for real-time MOT in complex environments.**

I. Introduction and Related Work

Multi-object tracking (MOT) is a critical function for autonomous systems operating in dynamic and unpredictable environments. It plays a central role in applications such as self-driving cars navigating busy streets and drones maneuvering through cluttered airspace. The ability to reliably estimate and forecast the trajectories of multiple agents is essential for ensuring safety, enabling collision avoidance, and supporting real-time decision-making. In this work, we focus on the various challenges of crowded environments including their unpredicted behaviour from a single image, the interactions between same-class agents, and the interaction between cross-classes agents in the scene. Namely, pedestrian-pedestrian interaction may be different than a cyclist-cyclist relationship, and different than pedestrian-cyclist relationship as they evolve over time.

A. Kalman Filtering

The Kalman Filter (KF), developed in the 1960s, remains widely used due to its mathematical elegance, computational efficiency, and optimality under linear-Gaussian assumptions. Specifically, it provides the best estimate in terms of minimizing the mean squared error (MSE) when both the system dynamics and observation models are linear, and noise is Gaussian. The KF further enables data fusion from multiple sensors [4]. Kim et al. use multiple Kalman filters within an integrated probabilistic data association framework to track several objects simultaneously. The filters combine complementary measurements—radial distance from radar and 2D image data from a CCD camera—demonstrating how Kalman filtering can enhance tracking robustness and accuracy in complex, real-world environments. Moreover, Kalman filter has proved to be reliable in a fully automatic MOT algorithm [5]. The system uses a fixed camera to monitor scenes and addresses common tracking challenges like occlusion, object merging, and splitting. A cost function is used to resolve object correspondences after splits. The method does not require manual initialization and is validated on human and vehicle video sequences, demonstrating effective tracking performance under complex, real-time conditions.

While KF is a powerful tool, the linear-Gaussian assumptions rarely hold in real-world tracking problems, where dynamics are often nonlinear and observations may be noisy or incomplete. The Extended Kalman Filter (EKF) addresses this by linearizing the nonlinear state transition and measurement functions around the current estimate. This enables tracking in a wider range of practical applications while preserving the recursive structure of the KF. However, applying the EKF requires the computation of Jacobian, which requires knowing the underlying state-space dynamical system with high confidence.

To avoid the pitfalls of computing Jacobians and obtaining explicit models of state transition and measurement functions, the Unscented Kalman Filter (UKF) can be utilized, which leverages the Sigma transformations process, explained in subsequent paragraphs. It has been successfully applied to MOT in complex 3D environments, such as those shown in the Waymo and nuScenes datasets [6]. The study demonstrated that the UKF can outperform traditional tracking methods such as AB3DMOT, particularly in scenarios with nonlinear dynamics.

The Sigma transformation in the UKF uses Sigma points which are a carefully chosen set of samples used to approximate the distribution of a random variable undergoing a nonlinear transformation. Unlike random sampling, they are

deterministically placed to capture the mean and covariance of the distribution, and are then propagated through the nonlinear function to estimate the transformed mean and covariance with higher accuracy than linearization (as in the EKF).

The most commonly used method for selecting sigma points in recent literature is by using Merwe-Scaled-Sigma-Points [10] [7], which deterministically chooses a set of $2n+1$ sigma points and their associated weights. Given the state dimension n, it produces one point at the mean and n symmetric pairs offset by the columns of $\sqrt{(n+\lambda)\,\Sigma}$, where Σ is the covariance and $\lambda = \alpha^2(n+\kappa) - n$. The three tuning hyper-parameters each have a clear role: α (typically a small number such as 10^{-3}) controls the overall spread of the points, β (set to 2 for optimal treatment of a Gaussian prior) injects prior knowledge about the distribution's peak-sharpness when computing the covariance weights, and κ (often 0) provides an additional degree of freedom to reduce higher-order errors. After the points are propagated through the nonlinear function, their weighted mean and covariance recover, to third-order accuracy, the transformed Gaussian, giving the UKF its robustness and accuracy without requiring explicit Jacobians.

Moreover, the UKF has often been integrated with neural networks to further enhance tracking performance. Newer research combines UKF with convolutional neural networks (CNN) [3] [8]. In these systems a CNN first extracts high-level measurements (such as object centroids, bounding-box keypoints, or even full 6-DoF poses) from raw images, while the UKF fuses those measurements over time, compensating for occlusion, missed detections, and nonlinear motion. This hybrid approach was shown to surpass both standalone UKF and conventional trackers in terms of MOT precision [3]. These extensions of the Kalman filter framework, show its potential to work in a much larger space, with weaker assumptions on the state and noise distributions.

B. Crowd Prediction

Understanding and forecasting the movement of crowds has long been a focus of research, with applications ranging from evacuation planning to autonomous navigation. The rise of autonomous vehicles, such as those developed by Waymo, has further amplified the need for accurate pedestrian trajectory prediction in dynamic, human-filled environments.

Neural networks, particularly recurrent architectures, have proven highly effective for this task. Long short-term memory (LSTM) networks have been shown to outperform traditional methods in predicting human motion in crowds, thanks to their ability to capture temporal dependencies and nonlinear dynamics [2].

In parallel, with hardware being more computationally powerful, traditional Kalman filtering methods have shown strong performance for real-time applications. The UKF, in particular, has become more applicable for tracking the real-time movements of multiple agents [1].

Recent work has explored hybrid models that combine LSTM-based neural networks with UKF filtering. In these approaches, the LSTM learns motion patterns or uncertainty representations, which the UKF uses to perform state estimation. This integration enables robust trajectory prediction even in the absence of continuous observations and the explicity dynamical model, given enough data, effectively combining the strengths of data-driven learning and probabilistic filtering [11].

II. Problem Definition

In this work, we propose a framework for learning trajectory predictions in uncertain environments by combining supervised learning with uncertainty-aware filtering techniques. Specifically, our goal is to learn a neural network that predicts both the dynamics model of a pedestrian system and the associated process noise. An Unscented Kalman Filter then samples from these learned components to perform state estimation.

Mathematically, we define the problem as follows: Let the ith agent state at discrete time step t be denoted by $\mathbf{x}_{t,i} \in \mathbb{R}^2$, representing the x-y position of the ith agent. The system evolves according to an unknown stochastic process governed by

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t) + \mathbf{w}_t, \tag{1}$$

where $f : \mathbb{R}^{\tau \times n} \to \mathbb{R}^n$ is an unknown, potentially nonlinear transition function with τ deciding the given number of previous states, and $\mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q}_t)$ is a zero-mean Gaussian process noise with unknown, time-varying covariance $\mathbf{Q}_t \in \mathbb{R}^{n \times n}$.

We observe the system through noisy measurements $\mathbf{z}_t \in \mathbb{R}^m$, modeled as

$$\mathbf{z}_t = h(\mathbf{x}_t) + \mathbf{v}_t, \tag{2}$$

where $h : \mathbb{R}^n \to \mathbb{R}^m$ is a known observation function, which we assume is given. And $\mathbf{v}_t \sim \mathcal{N}(0, \mathbf{R})$ is measurement noise with known covariance $\mathbf{R}$.

Our objective is to learn a parametric model f_θ through a neural network that, given a history of observed states and measurements $\mathcal{H}_t = \{\mathbf{x}_{\tau:t}, \mathbf{z}_{\tau:t}\}$, predicts both:

- the future state $\hat{\mathbf{x}}_{t+1} = f_\theta^{\text{pred}}(\mathcal{H}_t)$,
- and the corresponding process noise covariance $\hat{\mathbf{Q}}_t = f_\theta^{\text{noise}}(\mathcal{H}_t)$.

These learned quantities are then integrated into a UKF to perform uncertainty-aware state estimation.

A. Neural Network

For the first part of our project, we define one neural network with two objectives. The objectives are learning a generalized dynamics model to predict the agent's next state (given the previous 10 states) and estimating the process noise's positive definite covariance matrix.

Input

As input to our neural network we use the agent's state for the previous ten time steps, each state containing the 2D-coordinates (x, y). The history time $\tau = 10$ has empirically proven to provide best predictions over the next time step.

State Predictor

The first output of our neural network $f_\theta^{pred} : \mathbb{R}^{2\times 10} \to \mathbb{R}^2$, shown in (3), with parameters θ predicts the agent's next state $\hat{x}_{t+1}$ given the previous ten states $x_{\tau:t} = (x_{t-9}, \cdots, x_t)$.

$$\hat{x}_{t+1} = f_\theta^{pred}(x_{\tau:t}) \tag{3}$$

Noise Estimation

The next output from the neural network $f_\phi^{noise} : \mathbb{R}^{2\times 10} \to \mathbb{R}^{2\times 2}$ is parameterized by ϕ and the output is the computed process noise covariance $Q_t \succcurlyeq 0$.

Training

To train our neural network, we use a negative log-likelihood (NLL) loss with the $\log\det Q_t$ and the Mahalanobis distance, assuming Gaussian residuals, shown in (4).

$$\mathcal{L} = \sum_{t=1}^{T-1} \log\det Q_t + (x_{t+1} - f_\theta(\tilde{x}_t))^T Q_t^{-1}(x_{t+1} - f_\theta(\tilde{x}_t)))$$

where x_{t+1} is the ground truth state at timestep $t+1$.

B. Filtering via UKF

We then incorporate the learned model into an Unscented Kalman Filter. The system model is given by (4).

$$\bar{x}_{t+1} = f_\theta(\tilde{x}_t) + w_t, \quad w_t \sim \mathcal{N}(0, Q_t) \tag{4}$$

We also use measurements, with its model given by (5).

$$y_t = \bar{x}_t + v_t, \quad v_t \sim \mathcal{N}(0, R_t) \tag{5}$$

The UKF uses the Merwe-Scaled-Sigma-Point method to propagate the mean and covariance through the nonlinear dynamics, updating the state estimate and uncertainty at each time step, without requiring Jacobians.

C. Evaluation Metrics

We evaluate our method using Root Mean Squared Error (RMSE) between predicted state and true state in pixels.

III. Methods

A. Data

Due to the problem concerning mapping different agents around a university campus, we decided to use the Stanford Drone Dataset (SDD). SDD gives us access to sequential data in the form of $(x_1, x_2, \ldots, x_T)$, where $x_t \in \mathbb{R}^4$ denotes a specific agent's bounding box pixel coordinates in the camera frame at time t. Because SDD provides data in the form of bounding boxes, we convert these to 2D pixel positions in the center of the bounding box before inputting to our neural network. Our system makes noisy observations $y_t \in \mathbb{R}^2$ of the center of each agent.

All four videos, *video0* through *video3*, from the *Quad* section of SDD are used for training the neural network. For evaluation, *video0* from the section *Coupa* is used. This part of the data set was used because it contains the largest share of pedestrians and bikers [9].

B. Neural Network

The neural network described in Fig. 1 is trained using pedestrian trajectory data extracted from annotated video frames. Specifically, the training process models the transition dynamics and associated uncertainty of pedestrian motion. The training data consists of sequences of bounding box centroids from pedestrian tracks, where each training sample comprises a fixed-length history of positions and the corresponding next position as the target.

TABLE I
TABLE OF THE COMPONENTS OF THE NOISEMODEL ARCHITECTURE

Block	Input	Output
`DynamicsNet`	$\tilde{x}_t \in \mathbb{R}^{2\times 10}$	$\hat{x}_{t+1} \in \mathbb{R}^2$
`NoiseNet.encoder`	$\tilde{x}_t \in \mathbb{R}^{2\times 10}$	$h \in \mathbb{R}^{64}$
`NoiseNet.head`	$h \in \mathbb{R}^{64}$	$v \in \mathbb{R}^3$
`triangular`	$v \in \mathbb{R}^3$	$Q_t \in \mathbb{R}^{2\times 2}$

The overall `NoiseModel` architecture consists of several modular components, which are shown in Table I. `DynamicsNet` is a multilayer perceptron (MLP) that predicts residual velocity, taking the current state $\tilde{x}_t \in \mathbb{R}^{2\times 10}$ and outputting a next-state estimate $\hat{x}_{t+1} \in \mathbb{R}^2$. `NoiseNet.encoder` maps the state $\tilde{x}_t$ to a latent representation $h \in \mathbb{R}^{64}$ using a two-layer encoder with tanh as its activation function, which is shown in (6),

$$\tanh(\gamma) = \frac{e^\gamma - e^{-\gamma}}{e^\gamma + e^{-\gamma}} \tag{6}$$

where γ is the input.

The `NoiseNet.head` processes h to produce a 3-dimensional vector v parameterizing a lower-triangular matrix as shown in (7).

$$\mathbf{v} = [v_1, v_2, v_3] = \texttt{NoiseNet.head}(\mathbf{h}) \tag{7}$$

The `triangular` block then uses v to construct a positive semi-definite covariance matrix $Q_t \in \mathbb{R}^{2\times 2}$. This covariance matrix is parameterized via its lower-triangular Cholesky factor, and the diagonal elements are constrained to be positive using a softplus activation to ensure positive semi-definiteness. Firstly, $\mathbf{v}$ is used to construct a lower-triangular matrix $L_t \in \mathbb{R}^{2\times 2}$, as shown in (8). Softplus, which is shown in (9), is applied to the diagonal elements.

$$L_t = \begin{bmatrix} \text{softplus}(v_1) & 0 \\ v_2 & \text{softplus}(v_3) \end{bmatrix} \tag{8}$$

$$\text{softplus}(x) = \log(1 + e^x) \tag{9}$$

The process noise covariance matrix $Q_t \in \mathbb{R}^{2\times 2}$ is then computed via the equation shown in (10).

$$Q_t = L_t L_t^\top \tag{10}$$

With the final matrix product explicitly shown in (11).

$$Q_t = \begin{bmatrix} \text{softplus}(v_1)^2 & \text{softplus}(v_1)\cdot v_2 \\ \text{softplus}(v_1)\cdot v_2 & v_2^2 + \text{softplus}(v_3)^2 \end{bmatrix} \tag{11}$$

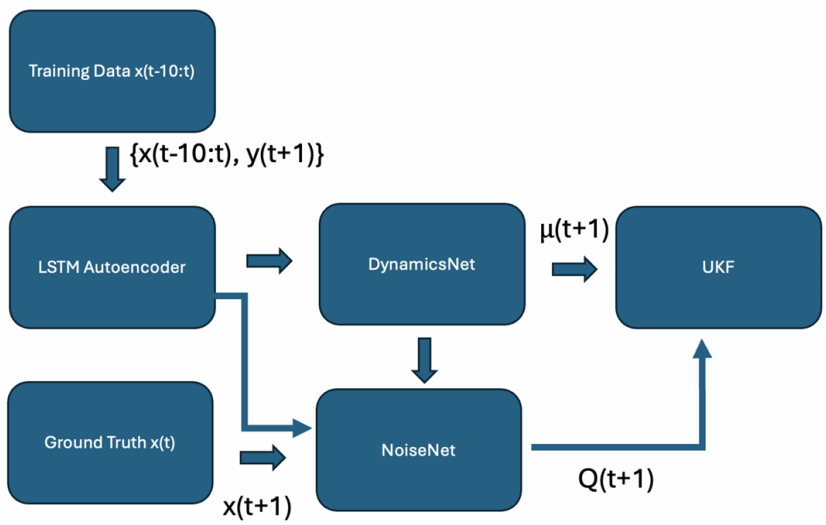

Fig. 1. Neural Network Architecture Block Diagram

This construction guarantees that Q_t is symmetric and positive semi-definite by design.

The model architecture includes three main components: an LSTM-based encoder that summarizes the history of positions into a fixed-size hidden vector, a dynamics head that predicts the next position, and a noise head that estimates a full covariance matrix.

The model is trained by minimizing the negative log-likelihood (NLL) of the ground-truth next state under the predicted Gaussian distribution. Specifically, the loss function penalizes both the Mahalanobis distance between the predicted mean and the actual next position, as well as the log-determinant of the predicted covariance matrix.

Training is conducted over 100 epochs using the Adam optimizer with a learning rate of 3×10^{-4} and mini-batches of size 512. Multiple annotated videos are automatically discovered from a root data directory, and their trajectory samples are combined into a single dataset.

C. Unscented Kalman Filter (UKF)

The Unscented Kalman Filter (UKF) was selected as the preferred method for predicting the final estimated state. Given that the dynamics are represented by a neural network, which makes computation of Jacobians impractical, the UKF presents a more suitable and natural alternative to the EKF. The UKF operates through the following key steps:

We begin by initializing the state as a Gaussian distribution:

Predict Step

$$X_0 \sim \mathcal{N}(\mu_{0|0}, \Sigma_{0|0}) \tag{12}$$

1. Sigma Point Generation (Merwe Scaling)

Using the Unscented Transform, we generate $2n+1$ sigma points from the current state estimate using the Merwe scaled sigma point method [10], this method provides a slightly more controlled approach to tighten / widen the spread of sigma points around the mean via the parameter α [7]:

$$\lambda = \alpha^2(n+\kappa) - n \tag{13}$$

$$\gamma = \sqrt{n+\lambda} \tag{14}$$

Let $A = \sqrt{\Sigma_{t-1|t-1}}$, computed via Cholesky decomposition. Then, the sigma points are:

$$X^{(0)}_{t-1|t-1} = \mu_{t-1|t-1} \tag{15}$$

$$X^{(i)}_{t-1|t-1} = \mu_{t-1|t-1} + \gamma A_{:,i-1}, \quad i = 1, \ldots, n \tag{16}$$

$$X^{(i+n)}_{t-1|t-1} = \mu_{t-1|t-1} - \gamma A_{:,i-1}, \quad i = 1, \ldots, n \tag{17}$$

The associated weights are:

$$w^{(0)}_m = \frac{\lambda}{n+\lambda} \tag{18}$$

$$w^{(0)}_c = \frac{\lambda}{n+\lambda} + (1 - \alpha^2 + \beta) \tag{19}$$

$$w^{(i)}_m = w^{(i)}_c = \frac{1}{2(n+\lambda)}, \quad i = 1, \ldots, 2n \tag{20}$$

Here:

- $\alpha \in (0, 1]$ determines the spread of the sigma points around the mean. A small α leads to tightly clustered points; typical values range from 10^{-3} to 0.1.
- $\kappa \in \mathbb{R}$ is a secondary scaling parameter. For Gaussian priors, a common choice is $\kappa = 0$ or $\kappa = 3 - n$.
- $\beta \geq 0$ incorporates prior knowledge about the distribution. For a Gaussian distribution, the optimal value is $\beta = 2$.

Here, $X^{(i)}_{t-1|t-1}$ denotes the i-th sigma point, and $w^{(i)}$ are the associated weights for computing the mean and covariance.

2. Propagation Through Dynamics

Each sigma point is propagated forward through the nonlinear system dynamics:

$$\bar{X}^{(i)}_{t|t-1} = f\left(X^{(i)}_{t-1|t-1}, u_{t-1}\right) \tag{21}$$

where u_{t-1} is the control input at time $t-1$. The weights $w^{(i)}$ remain unchanged in this step.

3. Predicted Mean

The predicted state mean is computed as the weighted average of the propagated sigma points:

$$\mu_{t|t-1} = \sum_{i=0}^{2n} w^{(i)} \bar{X}^{(i)}_{t|t-1} \tag{22}$$

4. Predicted Covariance

The predicted state covariance is computed from the deviations of the sigma points from the predicted mean, and the process noise covariance Q_{t-1} is added:

$$\Sigma_{t|t-1} = \sum_{i=0}^{2n} w^{(i)} \left(\bar{X}^{(i)}_{t|t-1} - \mu_{t|t-1}\right)\left(\bar{X}^{(i)}_{t|t-1} - \mu_{t|t-1}\right)^{\top} + Q_{t-1} \tag{23}$$

This concludes the prediction step of the Unscented Kalman Filter.

Update step: Given the predicted mean $\mu_{t|t-1}$ and covariance $\Sigma_{t|t-1}$ from the prediction step, we now incorporate the measurement to obtain the updated state estimate.

1. Sigma Point Generation

We generate sigma points from the predicted state:

$$\left\{X^{(i)}_{t|t-1},\, w^{(i)}\right\}_{i=0}^{2n} = \mathrm{UT}(\mu_{t|t-1}, \Sigma_{t|t-1}) \tag{24}$$

2. Predict Sigma Point Measurements

Each sigma point is passed through the nonlinear measurement function:

$$y^{(i)}_{t|t-1} = g\left(X^{(i)}_{t|t-1}\right) \tag{25}$$

3. Predicted Measurement Mean

The predicted measurement is computed as the weighted average of the transformed sigma points:

$$\hat{y}_{t|t-1} = \sum_{i=0}^{2n} w^{(i)} y^{(i)}_{t|t-1} \tag{26}$$

4. Measurement Covariance

The measurement covariance is computed as:

$$\Sigma_y = \sum_{i=0}^{2n} w^{(i)} \left(y^{(i)}_{t|t-1} - \hat{y}_{t|t-1}\right)\left(y^{(i)}_{t|t-1} - \hat{y}_{t|t-1}\right)^{\top} + R_t \tag{27}$$

where R_t is the measurement noise covariance.

5. Cross Covariance

The cross covariance between state and measurement is:

$$\Sigma_{xy} = \sum_{i=0}^{2n} w^{(i)} \left(X^{(i)}_{t|t-1} - \mu_{t|t-1}\right)\left(y^{(i)}_{t|t-1} - \hat{y}_{t|t-1}\right)^{\top} \tag{28}$$

The updated mean and covariance are computed via the Kalman gain:

$$\mu_{t|t} = \mu_{t|t-1} + \Sigma_{xy}\Sigma_y^{-1}\left(y_t - \hat{y}_{t|t-1}\right) \tag{29}$$

$$\Sigma_{t|t} = \Sigma_{t|t-1} - \Sigma_{xy}\Sigma_y^{-1}\Sigma_{xy}^{\top} \tag{30}$$

Return

Finally, we return $(\mu_{t|t}, \Sigma_{t|t})$.

To simulate the sensor imperfections we add a noise of $v_t \sim \mathcal{N}(0, 10I)$ to our measurements.

IV. Results

Testing the proposed framework on our chosen evaluation metric, the model is able to predict the dynamics of the agents down to the accuracies reported in Table I. The RMSE error reported is relative to the ground-truth labels provided by SDD and refer to the Kalman filter accuracies. The *video0* from the *Coupa* section of SDD, has 80 pedestrians and 38 bikers. Because of the significant discrepancy in accuracy between consistently moving and mostly static agents the results for each group is presented separately. The parenthesis gives the approximate number used for evaluation in each category. Another noteworthy result not shown in the summarizing table or plot is that the mean inference time is consistently below $0.006s$, highlighting the real-time applicability of this approach.

TABLE II
For approximate reference, a pedestrian fills a 70 x 30 pixel box, and a biker fills a 90 x 60 pixel box.

Agent Type / Trajectory Type (Num. Agents)	RMSE Error (Pixels)
Pedestrian / Walking (~ 50)	4.94
Pedestrian / Standing (~ 30)	771.48
Biker / Moving (~ 28)	18.19
Biker / Still (~ 10)	1107.64

Fig 2 shows the estimated versus true trajectory of 10 different agents throughout the video. Ground truth is shown as solid lines, estimated states shown in dotted lines, and are color coded by agents. For these agents that move in a semi-consistent behavior, it's evident that the proposed model predicts their trajectories well.

Focusing on one specific agent in the scene, Fig. 3 shows the trajectory of one pedestrian agent's trajectory with the corresponding 95% confidence ellipse provided by the UKF. The north-south direction of each trajectory is noticeably prone to larger uncertainty. This comes from the learned model, which has been trained on significantly more agents that move in the east-west horizontal direction.

Shown in Fig. 4 is a representative trajectory of a biker in the scene. We notice a similar behavior of the UKF in terms of performance compared to the pedestrian. This shows that

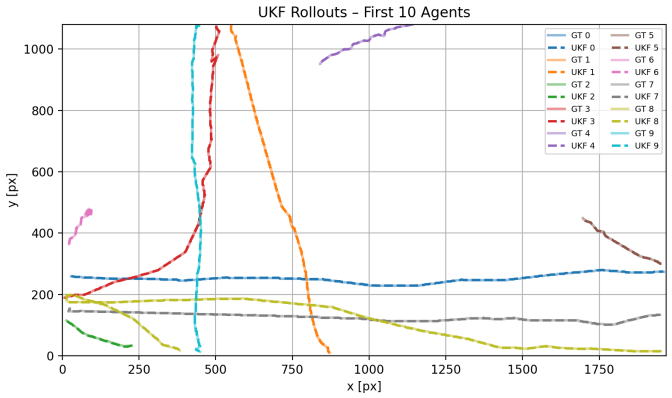

Fig. 2. Rollouts of the first ten agents in the Coupa dataset. Solid and dotted lines indicate each agent's ground truth and UKF-model-predicted trajectories respectively.

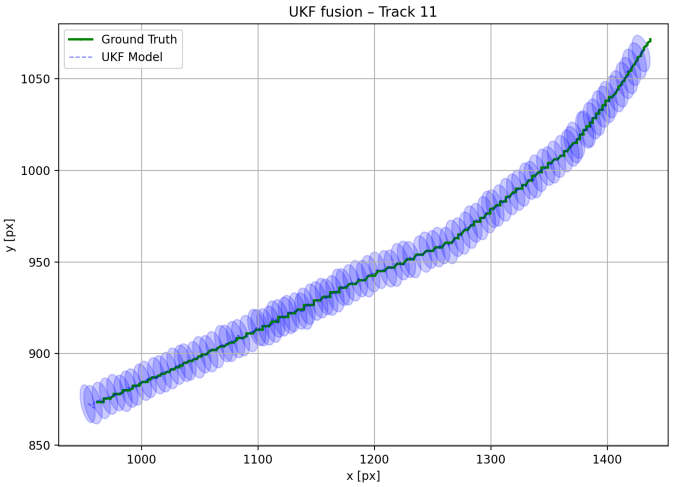

Fig. 3. Plot of Agent 11 (Pedestrian) trajectory, with uncertainty ellipses every 5 time steps.

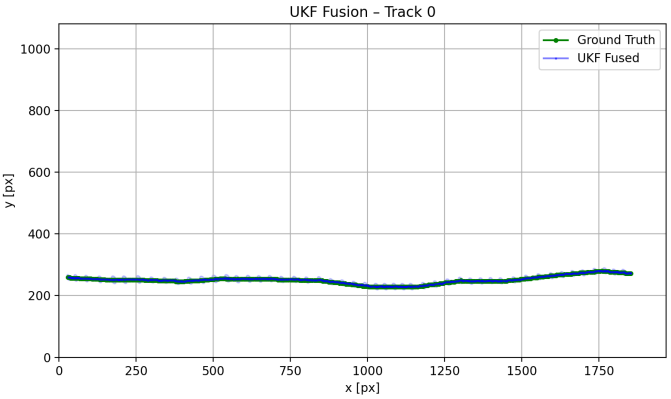

Fig. 4. Plot of Agent 0 (Biker) trajectory, with uncertainty ellipses every 5 time steps.

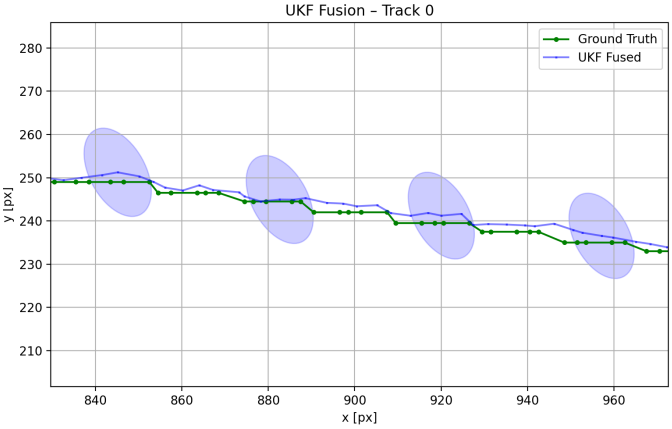

Fig. 5. Zoomed in Plot of Agent 0 (Biker) from Fig. 4.

the trained network is able to provide informative predictions about both types of agents, showing a potential for a flexible way to represent state dynamics, which has the potential to scale for many types of agents. Zooming into the trajectory from Fig. 5, again, showing the confidence ellipses plotted on top of the estimated states. We still see higher uncertainty along the vertical axis, which come from the same reason mentioned above being the neural network's lack of training data including more motion in the vertical axis of the image plane.

Note that the confidence ellipses are only plotted every couple of estimated states, to avoid cluttered plots.

V. Analysis and Discussion

The results show that for moving agents, the model performs well, providing predictions that can be considered to have relatively high accuracy in terms of RMSE. Qualitatively, the model tracks the ground truth quite well, as can be seen in Fig. 2. Furthermore, the average inference time being below 0.006 s for each agent, even with the limited performance of the available CPU, proves that it is also viable to run this model for online predictions.

The low errors for both pedestrians and bikers, as long as they are moving, demonstrate the model's ability to generalize between different types of agents. To further explore this, it would be interesting to see if the model can also generalize to other agent types, such as cars or even buses. We anticipate cars and buses to have more steady-state behavior and more consistent trajectories, implying that our model may perform better in predicting larger agent behavior.

A critical limitation of our model is observed in the RMSE differences between moving and stationary agents. For pedestrians, the model performs well during active walking, yielding an RMSE of 4.94 pixels, which suggests the model successfully learns short-term trajectory patterns for dynamic motion. However, when these same agents stand still for an extended period of time (at any point of the trajectory), the RMSE balloons to 771.48 pixels, indicating a severe degradation in predictive accuracy. This discrepancy is even more pronounced for bikers, where RMSE increases from 18.19 pixels during motion to 1107.64 pixels when still. Such high errors suggest the model becomes highly uncertain or erratic when tasked with forecasting static behavior over time.

These outcomes are likely exacerbated by the limited training data. Our dataset lacks sufficient coverage of diverse stationary behavior, especially for bikers, who are typically observed in motion in campus settings. The scarcity of such examples restricts the model's ability to learn meaningful priors for still states, especially over long horizons.

Another noteworthy factor is that the environment of our training and testing data is a university campus. This setting introduces inherent unpredictability. Students may stop suddenly to talk, respond to mobile devices, or change paths for other reasons. Modeling such unpredictable behavior requires a far more sophisticated model architecture and greater computing power to maintain real-time inference. A future direction would be to model such behavior far more robustly.

Lastly, measurement noise in simulation can amplify errors. Our current setup adds a synthetic perturbation to our measurements, simulating sensor imperfections. This may disproportionately affect stationary agents. A small error in one time step for a stationary agent translates into a large apparent movement, leading to major discrepancies in predicted versus actual positions, especially when the model wrongly interprets noise as intentional movement.

To address these problems with our model, future research should use more data, especially of stationary agents, but also from environments different from college campuses.

It could also be of interest to incorporate measurement noise into the neural network. Right now the model only predicts the process noise and then adds an artificial sensor noise. Allowing the neural network to predict both could potentially lead to more realistic performance overall while also improving predictions for static agents. However, when expanding the neural network, one would have to consider the increased inference time of computing yet another matrix since the ultimate goal of our model is to be viable to run online.

Furthermore, our project utilizes a UKF for the final state estimation, but other solutions are possible. Examples of avenues to explore include using a particle filter (PF) instead of a UKF or to further integrate the neural network by letting it give the final prediction step.

VI. Conclusion and Future Directions

Our trajectory prediction model shows strong performance for moving agents, with low RMSE for both walking pedestrians (4.94 px) and moving bikers (18.19 px). The model's low inference time ($< 0.006s$ per agent) also makes it suitable for real-time applications. These results suggest the model can generalize across agent types with different dynamics.

However, the model struggles significantly with stationary agents, yielding high RMSE values for both standing pedestrians (771.48 px) and still bikers (1107.64 px). This failure is likely due to, the model's inability to recognize or model stationary behavior because of a lack of stationary examples in the training dataset. Furthermore, the unpredictability of behaviors in a college campus environment and the compounding effect of simulated measurement noise, which can be misinterpreted as motion.

In conclusion, our model is able to accurately capture motion patterns for different agent types and is fast enough for online use. Still, the predictions for agents at rest are highly inaccurate, exposing a gap in the model's representation of stillness and intention. To address this we need an expanded dataset that includes more stationary agents and diverse environments. A larger dataset could also improve generalization and robustness. Another potential expansion would be allowing the neural network to jointly predict process and measurement noise instead of the current implementation which treats the latter as fixed perturbations. Although such an addition would have to be carefully balanced with computational efficiency to preserve the ability to run real-time inference. GitHub Repository

References

[1] Robert Clay, Le-Minh Kieu, Jonathan A. Ward, Alison Heppenstall, and Nick Malleson. Towards real-time crowd simulation under uncertainty using an agent-based model and an unscented kalman filter. In Yves Demazeau, Tom Holvoet, Juan M. Corchado, and Stefania Costantini, editors, *Advances in Practical Applications of Agents, Multi-Agent Systems, and Trustworthiness. The PAAMS Collection*, pages 68–79, Cham, 2020. Springer International Publishing.

[2] Akif Hacinecipoglu, E. Ilhan Konukseven, and A. Bugra Koku. Multiple human trajectory prediction and cooperative navigation modeling in crowded scenes. *Intelligent Service Robotics*, 13(4):479–493, October 2020.

[3] Thirumalai Jaganathan, Anandan Panneerselvam, and Senthil Kumar Kumaraswamy. Object detection and multi-object tracking based on optimized deep convolutional neural network and unscented kalman filtering. *Concurrency and Computation: Practice and Experience*, 34(25):e7245, 2022.

[4] Du Yong Kim and Moongu Jeon. Data fusion of radar and image measurements for multi-object tracking via kalman filtering. *Information Sciences*, 278:641–652, 2014.

[5] Xin Li, Kejun Wang, Wei Wang, and Yang Li. A multiple object tracking method using kalman filter. In *The 2010 IEEE International Conference on Information and Automation*, pages 1862–1866, 2010.

[6] Meng Liu, Jianwei Niu, and Yu Liu. Ukf-mot: An unscented kalman filter-based 3d multi-object tracker. *CAAI Transactions on Intelligence Technology*, 9(4):1031–1041, 2024.

[7] Rudolph Merwe and Eric Wan. Sigma-point kalman filters for probabilistic inference in dynamic state-space models. *Proceedings of the Workshop on Advances in Machine Learning*, 06 2003.

[8] Tae Ha Park and Simone D'Amico. Adaptive neural-network-based unscented kalman filter for robust pose tracking of noncooperative spacecraft. *Journal of Guidance, Control, and Dynamics*, 46(9):1671–1688, 2023.

[9] Alexandre Robicquet, Amir Sadeghian, Alexandre Alahi, and Silvio Savarese. Learning social etiquette: Human trajectory prediction in crowded scenes. In *European Conference on Computer Vision (ECCV)*, pages 549–565. Springer, 2016.

[10] Eric A Wan and Rudolph Van Der Merwe. The unscented kalman filter for nonlinear estimation. In *Proceedings of the IEEE 2000 adaptive systems for signal processing, communications, and control symposium (Cat. No. 00EX373)*, pages 153–158. Ieee, 2000.

[11] Jing Zhang, YingNian Wu, and Shuai Jiao. Research on trajectory tracking algorithm based on lstm-ukf. In *2021 7th IEEE International Conference on Network Intelligence and Digital Content (IC-NIDC)*, pages 61–65, 2021.